

Practical 9

Task1: Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite)

Code:

```
rohita@DESKTOP-1UHU58C: ~  
GNU nano 8.7.1 p91.c *  
#include <stdio.h>  
#include <stdlib.h>  
#include <fcntl.h>  
#include <unistd.h>  
#include <time.h>  
  
#define BUFFER_SIZE 4096  
  
void low_level_copy(const char *src, const char *dest) {  
    int in = open(src, O_RDONLY);  
    int out = open(dest, O_WRONLY | O_CREAT | O_TRUNC, 0644);  
    char buffer[BUFFER_SIZE];  
    ssize_t n;  
  
    if (in < 0 || out < 0) {  
        perror("open");  
        exit(1);  
    }  
  
    lseek(in, 0, SEEK_SET);  
  
    while ((n = read(in, buffer, BUFFER_SIZE)) > 0)  
        write(out, buffer, n);  
  
    close(in);  
    close(out);  
}  
  
void stdio_copy(const char *src, const char *dest) {  
    FILE *in = fopen(src, "rb");  
    FILE *out = fopen(dest, "wb");  
    char buffer[BUFFER_SIZE];  
    size_t n;  
  
    if (!in || !out) {  
        perror("fopen");  
        exit(1);  
    }  
  
    while ((n = fread(buffer, 1, BUFFER_SIZE, in)) > 0)  
        fwrite(buffer, 1, n, out);  
  
    fclose(in);  
    fclose(out);  
}
```

```

int main() {
    clock_t start, end;
    double t1, t2;

    low_level_copy("source.txt", "lowlevel.txt");

    start = clock();
    low_level_copy("source.txt", "lowlevel.txt");
    end = clock();
    t1 = (double)(end - start) / CLOCKS_PER_SEC;

    start = clock();
    stdio_copy("source.txt", "stdio.txt");
    end = clock();
    t2 = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Low-level copy completed.\n");
    printf("Standard library copy completed.\n");
    printf("Low-level I/O time: %.6f seconds\n", t1);
    printf("Standard I/O time: %.6f seconds\n", t2);

    return 0;
}

```

Output:

```

Low-level copy completed.
Standard library copy completed.
Low-level I/O time: 0.000581 seconds
Standard I/O time: 0.000631 seconds
rohita@DESKTOP-1UHU58C:~$

```

Observation: The file was successfully copied using both low-level system calls and standard library functions. The execution times were measured and compared, demonstrating the difference between system-level and standard I/O operations.

Task 2: Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

Code:

```
rohita@DESKTOP-1UHU58C: ~  
GNU nano 8.7.1 p92.c *  
include <stdio.h>  
include <fcntl.h>  
include <unistd.h>  
include <stdlib.h>  
  
int main() {  
    int input, output;  
    char buffer[100];  
  
    input = open("input.txt", O_RDONLY);  
    if (input < 0) {  
        perror("input.txt");  
        return 1;  
    }  
  
    output = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);  
    if (output < 0) {  
        perror("output.txt");  
        return 1;  
    }  
  
    dup2(input, STDIN_FILENO);  
    dup2(output, STDOUT_FILENO);  
  
    close(input);  
    close(output);  
  
    printf("Reading from redirected input:\n");  
  
    while (fgets(buffer, sizeof(buffer), stdin) != NULL)  
        printf("%s", buffer);  
  
    return 0;  
}
```

Output:

```
rohita@DESKTOP-1UHU58C:~$ gcc p92.c  
rohita@DESKTOP-1UHU58C:~$ ./a.out  
rohita@DESKTOP-1UHU58C:~$ cat output.txt  
Reading from redirected input:  
Hello from input file
```

Observation: dup2() successfully redirected standard input from input.txt and standard output to output.txt, demonstrating the basic mechanism used by shells for I/O redirection.